

Lab7：进程间通信

本章完成的工作

本章主要完成了以下任务：

- 1. 理解管道（Pipe）的概念和实现原理
- 2. 理解"一切皆文件"的 Unix 设计哲学
- 3. 分析文件描述符表的设计和作用
- 4. 理解 fork 如何继承文件描述符实现父子进程通信
- 5. 运行管道测试程序，验证进程间通信功能

本章没有编程作业，代码由清华提供。问答作业需要写在 ch6 报告里一并提交。

报告结构说明

本报告的结构安排及与清华指导书的对比：

清华指导书小节	本报告对应小节	差异说明
引言/本章导读	第二节	从"为什么需要进程间通信"的问题出发理解
文件系统扩充	第三节	重点理解"一切皆文件"的设计思想
pipe管道的实现	第四节	用结构图展示环形缓冲区的工作原理
pipe相关系统调用	第五节	分析系统调用的参数和返回值
进程通讯与fork	第六节	理解 fork 继承文件描述符的机制

本报告的特点：

- 1. 不是照搬指导书内容，而是体现学习探索过程
- 2. 增加了两张原创结构图帮助理解
- 3. 从实际问题出发理解设计决策

一、实验环境与运行

1.1 代码目录

```
cd /桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch7（其他同学可根据实际路径）
```

本章新增的关键文件：

- `os/pipe.c`：管道的核心实现
- `os/file.c`：文件抽象层

- `os/file.h`: 文件和管道的数据结构定义

1.2 运行命令

```
make clean
make user BASE=1 CHAPTER=7
make run
# 在 shell 中执行:
>> ch7b_usertest
```

1.3 运行结果

```
>> ch7b_usertest
Userests: Running ch2b_hello_world
Hello world from user mode program!
Test hello_world OK!
Userests: Test ch2b_hello_world in Process 3 exited with code 0
Userests: Running ch2b_power
3^10000=5079
3^20000=8202
3^30000=8824
3^40000=5750
3^50000=3824
3^60000=8516
3^70000=2510
3^80000=9379
3^90000=2621
3^100000=2749
Test power OK!
Userests: Test ch2b_power in Process 4 exited with code 0
Userests: Running ch3b_sleep
get_time OK! 9156
Test sleep OK!
Userests: Test ch3b_sleep in Process 5 exited with code 0
Userests: Running ch3b_sleep1
current time_msec = 12159
time_msec = 12260 after sleeping 100 ticks, delta = 101ms!
Test sleep1 passed!
Userests: Test ch3b_sleep1 in Process 6 exited with code 0
Userests: Running ch3b_yield0
AAAAAAAAAA [1/5]
AAAAAAAAAA [2/5]
AAAAAAAAAA [3/5]
AAAAAAAAAA [4/5]
AAAAAAAAAA [5/5]
Test write A OK!
Userests: Test ch3b_yield0 in Process 7 exited with code 0
Userests: Running ch3b_yield1
CCCCCCCCCC [1/5]
CCCCCCCCCC [2/5]
CCCCCCCCCC [3/5]
CCCCCCCCCC [4/5]
```

```
CCCCCCCCC [5/5]
Test write C OK!
Userstests: Test ch3b_yield1 in Process 8 exited with code 0
Userstests: Running ch3b_yield2
BBBBBBBBBB [1/5]
BBBBBBBBBB [2/5]
BBBBBBBBBB [3/5]
BBBBBBBBBB [4/5]
BBBBBBBBBB [5/5]
Test write B OK!
Userstests: Test ch3b_yield2 in Process 9 exited with code 0
Userstests: Running ch5b_exit
I am the parent. Forking the child...
I am parent, fork a child pid 11
I am the parent, waiting now..
I am the child.
waitpid 11 ok.
exit pass.
Userstests: Test ch5b_exit in Process 10 exited with code 0
Userstests: Running ch5b_getpid
Test getppid OK!
Test getpid OK! pid = 12, ppid = 2
Userstests: Test ch5b_getpid in Process 12 exited with code 0
Userstests: Running ch5b_forktest0
sys_wait without child process test passed!
parent start, pid = 14!
ready waiting on parent process!
hello child process!
child process pid = 15, exit code = 100
forktest0 pass.
Userstests: Test ch5b_forktest0 in Process 14 exited with code 0
Userstests: Running ch5b_forktest1
forked child pid = 17
forked child pid = 18
forked child pid = 19
forked child pid = 20
forked child pid = 21
forked child pid = 22
forked child pid = 23
forked child pid = 24
forked child pid = 25
forked child pid = 26
forked child pid = 27
forked child pid = 28
forked child pid = 29
forked child pid = 30
forked child pid = 31
forked child pid = 32
forked child pid = 33
forked child pid = 34
forked child pid = 35
forked child pid = 36
forked child pid = 37
```

```
forked child pid = 38
forked child pid = 39
forked child pid = 40
forked child pid = 41
forked child pid = 42
forked child pid = 43
forked child pid = 44
forked child pid = 45
forked child pid = 46
forked child pid = 47
forked child pid = 48
forked child pid = 49
forked child pid = 50
forked child pid = 51
forked child pid = 52
forked child pid = 53
forked child pid = 54
forked child pid = 55
forked child pid = 56
I am child 0
I am child 1
I am child 3
I am child 4
I am child 5
I am child 6
I am child 7
I am child 8
I am child 9
I am child 10
I am child 11
I am child 13
I am child 14
I am child 15
I am child 16
I am child 17
I am child 18
I am child 19
I am child 20
I am child 22
I am child 23
I am child 24
I am child 26
I am child 27
I am child 29
I am child 30
I am child 31
I am child 32
I am child 33
I am child 34
I am child 35
I am child 36
I am child 38
I am child 39
```

```
I am child 2
I am child 12
I am child 21
I am child 25
I am child 28
I am child 37
forktest1 pass.
Userstests: Test ch5b_forktest1 in Process 16 exited with code 0
Userstests: Running ch5b_forktest2
Hello world from user mode program!
Test hello_world OK!
Hello world from user mode program!
Test hello_world OK!
Hello world from user mode program!
Test hello_world OK!
Hello world from user mode program!
Test hello_world OK!
Hello world from user mode program!
Test hello_world OK!
forktest2 test passed!
Userstests: Test ch5b_forktest2 in Process 57 exited with code 0
Userstests: Running ch5b_exit
I am the parent. Forking the child...
I am parent, fork a child pid 64
I am the parent, waiting now..
I am the child.
waitpid 64 ok.
exit pass.
Userstests: Test ch5b_exit in Process 63 exited with code 0
Userstests: Running ch6b_filetest
open OK, fd = 3
write over.
hello world!
read over.
filetest passed.
Userstests: Test ch6b_filetest in Process 65 exited with code 0
Userstests: Running ch6b_exec
argc = 6
argv[0] = (*o*)
argv[1] = (>.<)
argv[2] = (0.0)
argv[3] = (QwQ)
argv[4] = orz
argv[5] = 没有了呀
Userstests: Test ch6b_exec in Process 67 exited with code 0
Userstests: Running ch7b_pipetest
[parent] read end = 0x0000000000000003, write end = 0x0000000000000004
[parent] close read end
[parent] write over
[child] close write end
[chile] read over
Read OK, child process exited!
pipetest passed!
```

```
Usertests: Test ch7b_pipetest in Process 68 exited with code 0
ch7b Usertests passed!
Shell: Process 2 exited with code 0
>>
```

二、为什么需要进程间通信？

2.1 从实际问题出发

在学习前几章时，我注意到每个进程都是"孤立"的：

- 进程只能通过键盘获取输入 (stdin)
- 进程只能向屏幕输出结果 (stdout)
- 进程之间无法直接交换数据

这让我想到一个问题：如果我想让一个程序的输出作为另一个程序的输入，该怎么办？

比如在 Linux 终端中，我们经常使用这样的命令：

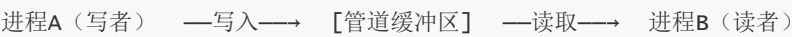
```
cat file.txt | wc -l      # 统计文件行数
ls -la | grep ".c"       # 过滤出 .c 文件
```

这里的 `|` 符号就是管道，它把前一个命令的输出"接"到后一个命令的输入。

2.2 管道解决了什么问题？

刚开始我以为进程间通信很复杂，可能需要共享内存或者网络通信之类的机制。但管道的设计让我眼前一亮：

管道的本质：就是一块内核管理的缓冲区，一个进程往里写，另一个进程从里面读。



这样简单的设计，却能够实现强大的进程间协作功能。

2.3 管道的特点

通过阅读指导书，我总结出管道的几个关键特点：

特点	说明
单向通信	数据只能从写端流向读端
字节流	传输的是无结构的字节序列
有缓冲	有固定大小的缓冲区（本实验为 512 字节）
阻塞语义	读空管道会阻塞，写满管道也会阻塞

三、"一切皆文件"的设计哲学

3.1 初见困惑

指导书提到 Unix 的设计哲学是"一切皆文件"（Everything is a file），刚开始我不太理解：管道明明是一个内存缓冲区，怎么能说是"文件"呢？

3.2 理解统一抽象的好处

通过阅读 `file.h` 中的代码，我理解了这个设计的精妙之处：

```
struct file {
    enum { FD_NONE = 0, FD_PIPE, FD_INODE, FD_STDIO } type;
    int ref;           // 引用计数
    char readable;     // 是否可读
    char writable;     // 是否可写
    struct pipe *pipe; // 如果是管道类型
    struct inode *ip;  // 如果是普通文件类型
    uint off;          // 文件偏移量
};
```

关键洞察：不管是键盘输入、屏幕输出、管道还是磁盘文件，对于应用程序来说，操作方式都是一样的——通过 `read()` 和 `write()` 系统调用！

这带来了巨大的好处：

- 1. **接口统一：**应用程序不需要为不同的 I/O 设备学习不同的 API
- 2. **可组合性：**可以轻松地将程序的输入输出重定向
- 3. **简化开发：**程序员只需要关心"读"和"写"，不需要关心底层是什么设备

3.3 文件描述符的作用

每个进程都有一个文件描述符表，用于管理当前打开的"文件"：

文件描述符	默认对应	说明
0	stdin	标准输入（键盘）
1	stdout	标准输出（屏幕）
2	stderr	标准错误（屏幕）
3+	用户打开的文件/管道	由系统分配

当调用 `sys_pipe()` 创建管道时，内核会分配两个新的文件描述符（比如 3 和 4），分别对应管道的读端和写端。

四、管道的实现原理

4.1 管道的数据结构

```
#define PIPESIZE 512

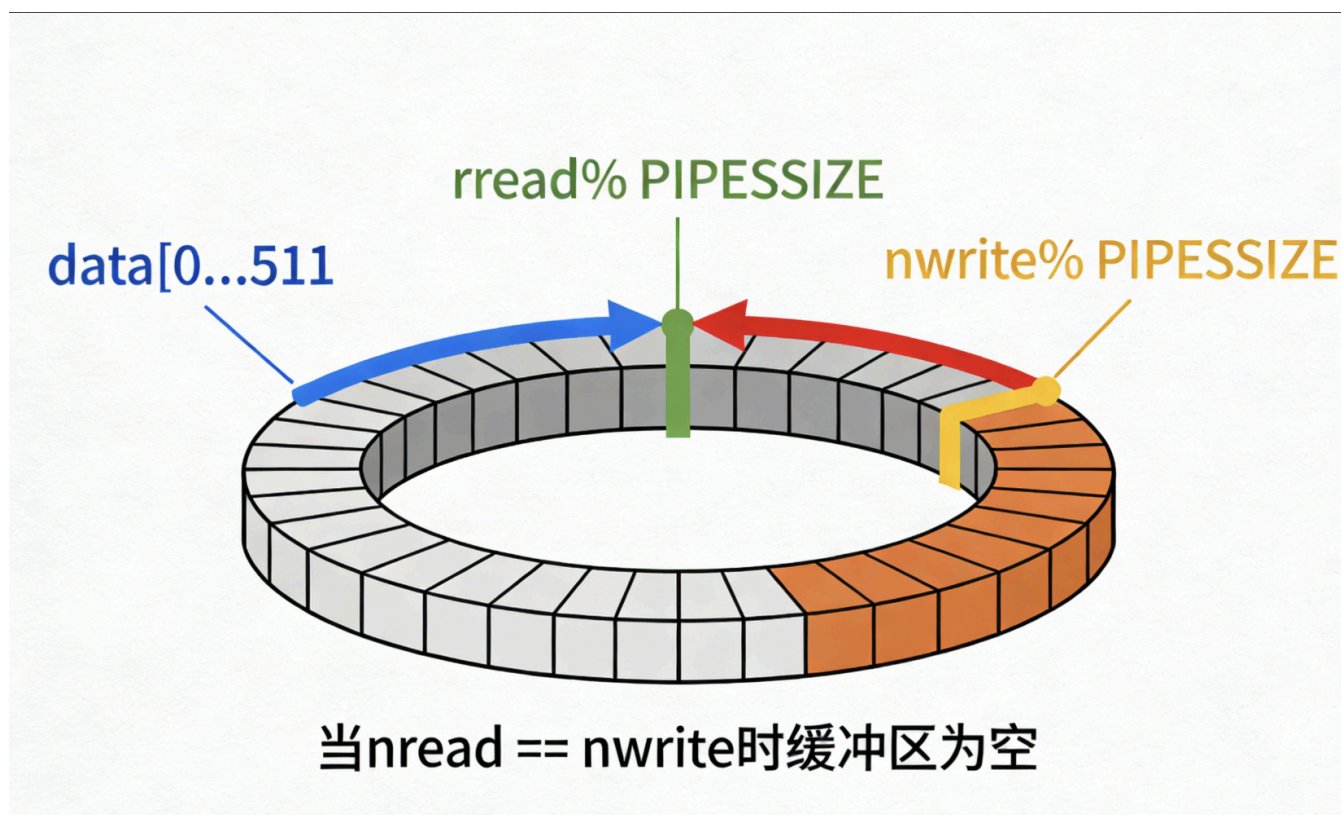
struct pipe {
    char data[PIPESIZE]; // 环形缓冲区
    uint nread;           // 已读取的字节数
    uint nwrite;          // 已写入的字节数
    int readopen;         // 读端是否打开
    int writeopen;        // 写端是否打开
};
```

4.2 环形缓冲区的工作原理

刚开始看到 `nread` 和 `nwrite` 这两个变量时，我有些困惑：为什么不直接用数组下标？

通过阅读 `piperead()` 和 `pipewrite()` 的代码，我理解了这是一个**环形缓冲区**的设计：

【结构图1：环形缓冲区示意图】



为什么用累计计数而不是数组下标？

我思考后理解到：

- `nread` 和 `nwrite` 是累计值，可以无限增长
- 实际的数组下标通过取模运算得到：`nread % PIPESIZE`
- 通过 `nwrite - nread` 可以直接得到缓冲区中的数据量
- 这种设计避免了复杂的边界判断

4.3 管道的阻塞行为

管道的读写有特殊的阻塞语义：

读操作 (piperead) :

```
while(pi->nread == pi->nwrite) { // 缓冲区为空
    if(pi->writeopen)
        yield(); // 写端还开着，等待数据
    else
        return -1; // 写端已关闭，返回错误
}
```

写操作 (pipewrite) :

```
if(pi->nwrite == pi->nread + PIPESIZE) { // 缓冲区已满
    yield(); // 等待读者腾出空间
}
```

这个设计让我想到了生产者-消费者模型：写者是生产者，读者是消费者，管道缓冲区是共享的仓库。

五、管道相关的系统调用

5.1 sys_pipe：创建管道

```
uint64 sys_pipe(uint64 fdarray)
```

参数：用户空间的数组地址，用于接收两个文件描述符

返回值：成功返回 0，失败返回 -1

执行流程：

1. 分配两个 `struct file` 结构
2. 调用 `pipealloc()` 创建管道并关联两个文件
3. 分配两个文件描述符 (fd0 用于读，fd1 用于写)
4. 通过 `copyout()` 将 fd 写回用户空间

5.2 sys_close：关闭文件描述符

```
uint64 sys_close(int fd)
```

参数：要关闭的文件描述符

关键逻辑：

- 如果是管道类型，调用 `pipeclose()`
- 使用引用计数管理：只有当 `ref` 减到 0 时才真正释放资源

- 管道需要读端和写端都关闭后才释放缓冲区

5.3 sys_read 和 sys_write 的修改

本章对读写系统调用进行了扩展，根据文件描述符的类型选择不同的处理方式：

```
uint64 sys_write(int fd, uint64 va, uint64 len) {
    if (fd == STDOUT || fd == STDERR) {
        return console_write(va, len); // 屏幕输出
    }
    // ...
    if (f->type == FD_PIPE) {
        return pipewrite(f->pipe, va, len); // 管道写入
    }
}
```

这体现了"一切皆文件"的设计：同一个 `write()` 系统调用，可以写屏幕，也可以写管道。

六、fork 与管道的配合

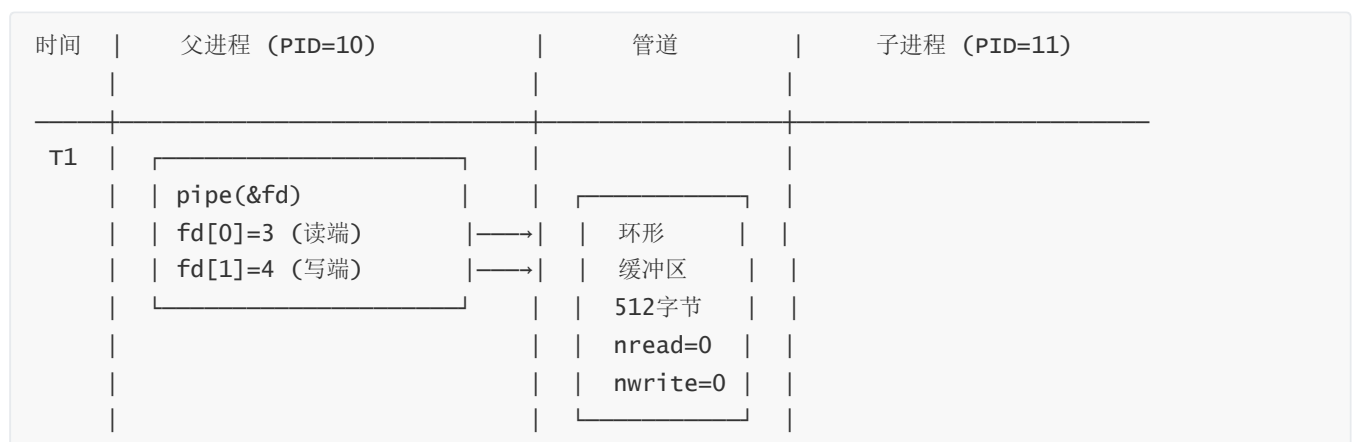
6.1 fork 为什么能实现进程间通信？

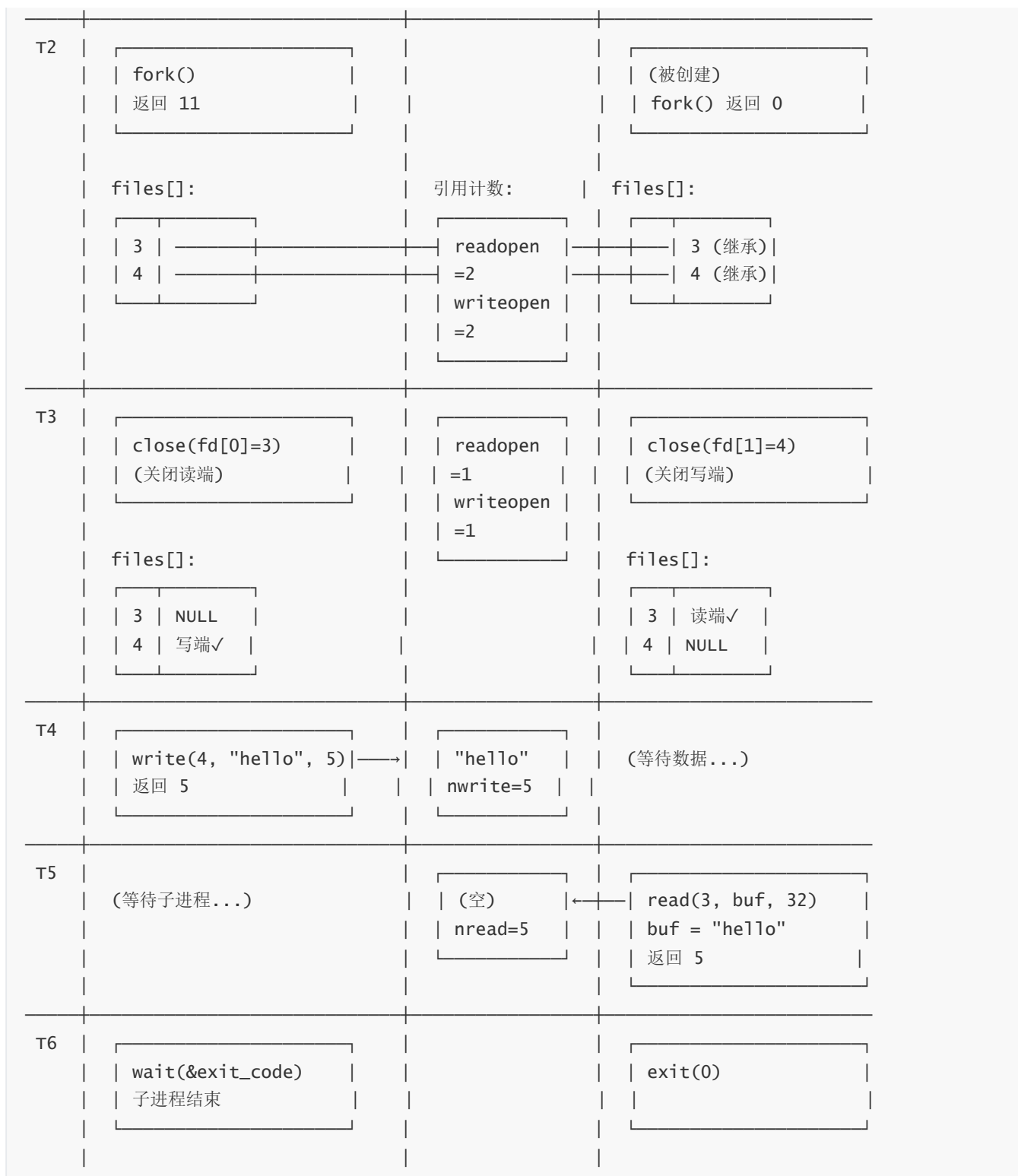
这是本章最让我感到精妙的地方。在 Lab5 学习 fork 时，我只知道它会复制进程的内存。现在才理解，fork 还会**复制文件描述符表**：

```
int fork() {
    // ...
    for(int i = 3; i < FD_MAX; ++i)
        if(p->files[i] != 0 && p->files[i]->type != FD_NONE) {
            p->files[i]->ref++; // 增加引用计数
            np->files[i] = p->files[i]; // 子进程继承同一个文件指针
        }
    // ...
}
```

关键洞察：父子进程的文件描述符指向**同一个**内核文件结构，所以它们可以通过管道通信！

6.2 管道通信的典型模式





关键点标注 (在图旁边用文字框说明) :

1. T1-创建管道:

- pipe() 返回两个 fd: 3(读) 和 4(写)
- 管道缓冲区 512 字节
- readopen=1, writeopen=1

2. T2-fork 继承:

- 子进程继承父进程的 files[] 数组

- 两个进程的 fd[3] 和 fd[4] 指向**同一个**内核 file 结构
- 引用计数变为 2

3. T3-各自关闭不需要的端：

- 父进程只写，关闭读端
- 子进程只读，关闭写端
- 这是管道使用的标准模式

4. T4-T5 数据传输：

- 父进程 write → 管道缓冲区 → 子进程 read
- 数据通过内核缓冲区传递

6.3 为什么 exec 不影响管道通信？

指导书提到：exec 不会关闭已打开的文件描述符。

这意味着：

```
if (fork() == 0) {
    // 子进程
    exec("/bin/some_program");
    // exec 之后，管道的文件描述符仍然有效！
}
```

这个设计使得 shell 的管道功能成为可能：`cmd1 | cmd2` 可以让两个完全不同的程序通过管道通信。

6.4 测试程序分析

```
// user/src/ch7b_pipetest.c
char STR[] = "hello pipe!";

int main() {
    uint64 pipe_fd[2];
    pipe(&pipe_fd);          // 创建管道

    if (fork() == 0) {
        // 子进程：读取数据
        char buffer[32 + 1];
        read(pipe_fd[0], buffer, 32);
        assert(strncmp(buffer, STR, strlen(STR)) == 0);
        exit(0);
    } else {
        // 父进程：写入数据
        write(pipe_fd[1], STR, strlen(STR));
        wait(&exit_code);
    }
    return 0;
}
```

这个测试程序验证了：

1. `pipe()` 能正确创建管道
2. `fork()` 后父子进程共享管道
3. 父进程写入的数据能被子进程读取

七、问答作业

注：根据清华指导书要求，问答作业需要写在 ch6 的报告里一并提交。这里先写出答案，后续整合到 Lab6 报告中。

问题1：举出使用 pipe 的一个实际应用的例子

答案：

在 Linux 终端中，管道最常见的应用是将多个命令组合起来完成复杂任务：

```
# 统计当前目录下 .c 文件的行数
cat *.c | wc -l

# 查找包含 "error" 的日志行
cat /var/log/syslog | grep "error"

# 统计文件行数 (cat + wc)
cat file.txt | wc -l
```

工作原理：

- `cat` 命令的标准输出被重定向到管道的写端
- `wc -l` 命令的标准输入被重定向到管道的读端
- Shell 通过 `fork + pipe + exec` 实现这一功能

问题2：设计一个更易用的多进程通信机制

答案：

管道的局限性：

- 只能在有亲缘关系的进程间使用（需要 `fork` 继承）
- 只能单向通信
- 需要为每对进程建立一个管道

设计一个**命名消息队列**机制：

```
// 创建一个命名消息队列
int mq = mq_create("/my_queue", MAX_MESSAGES);

// 任何知道队列名称的进程都可以发送消息
mq_send("/my_queue", message, len);

// 任何知道队列名称的进程都可以接收消息
mq_receive("/my_queue", buffer, max_len);
```

优点:

1. **无需亲缘关系**: 任何进程只要知道队列名称就能通信
2. **多对多通信**: 多个发送者, 多个接收者
3. **消息边界**: 保留消息的边界, 不是字节流
4. **持久性**: 队列可以在进程退出后继续存在

这类似于 POSIX 消息队列或 System V 消息队列的设计思想。

八、实验总结

8.1 完成情况

- ✓ 理解管道的基本概念和工作原理
- ✓ 理解"一切皆文件"的设计哲学
- ✓ 分析管道的数据结构和环形缓冲区实现
- ✓ 理解 sys_pipe 和 sys_close 系统调用
- ✓ 理解 fork 如何继承文件描述符实现进程间通信
- ✓ 完成问答作业

8.2 收获与体会

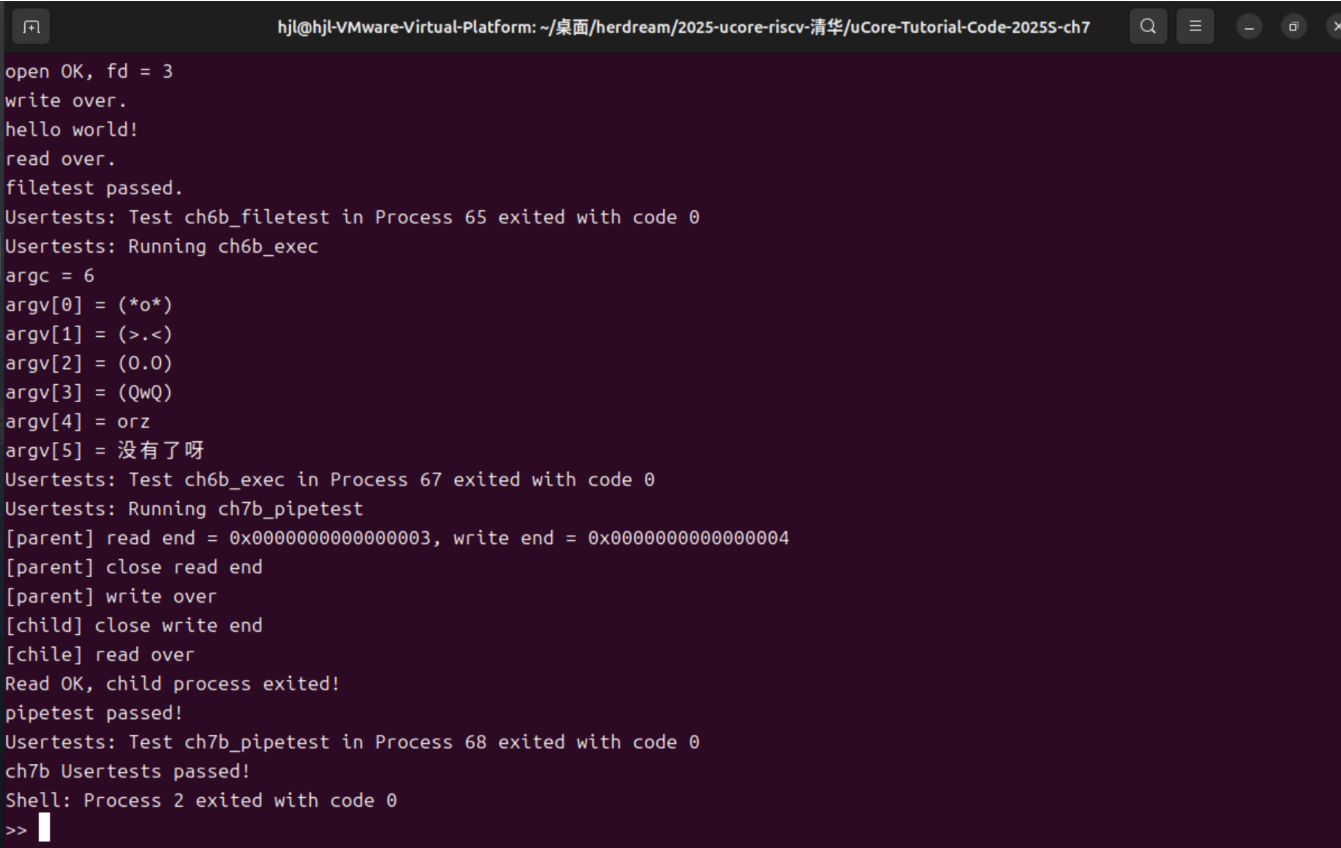
1. **设计之美**: 管道的设计非常简洁——就是一块缓冲区加上读写两端的文件描述符, 却能实现强大的进程间通信功能。
2. **统一抽象的力量**: "一切皆文件"让不同类型的 I/O 资源有了统一的接口, 大大简化了系统设计和应用开发。
3. **fork 的深层含义**: 之前只理解 fork 复制内存, 现在理解到它还复制文件描述符表, 这是实现进程间通信的基础。
4. **环形缓冲区**: 通过 nread 和 nwrite 两个累计计数器实现环形缓冲区, 避免了复杂的边界判断, 是一个很巧妙的设计。

8.3 本章涉及的文件

文件	作用
os/file.h	定义 file 和 pipe 结构体
os/file.c	文件操作的实现 (filealloc、fileclose 等)

文件	作用
os/pipe.c	管道的核心实现（pipealloc、piperead、pipewrite）
os/syscall.c	系统调用处理（sys_pipe、sys_close、扩展的 sys_read/write）
os/proc.c	fork 中增加文件描述符继承，freeproc 中增加文件清理

九、验证截图

A terminal window with a dark purple background and light green text. The window title is 'hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch7'. The output shows a series of test results: 'open OK, fd = 3', 'write over.', 'hello world!', 'read over.', 'filetest passed.', 'Ustests: Test ch6b_filetest in Process 65 exited with code 0', 'Ustests: Running ch6b_exec', 'argc = 6', 'argv[0] = (*o*)', 'argv[1] = (>.<)', 'argv[2] = (0.0)', 'argv[3] = (QwQ)', 'argv[4] = orz', 'argv[5] = 没有了呀', 'Ustests: Test ch6b_exec in Process 67 exited with code 0', 'Ustests: Running ch7b_pipetest', '[parent] read end = 0x0000000000000003, write end = 0x0000000000000004', '[parent] close read end', '[parent] write over', '[child] close write end', '[chile] read over', 'Read OK, child process exited!', 'pipetest passed!', 'Ustests: Test ch7b_pipetest in Process 68 exited with code 0', 'ch7b Ustests passed!', 'Shell: Process 2 exited with code 0', and '>>' at the prompt.

```
hjl@hjl-VMware-Virtual-Platform: ~/桌面/herdream/2025-ucore-riscv-清华/uCore-Tutorial-Code-2025S-ch7
open OK, fd = 3
write over.
hello world!
read over.
filetest passed.
Ustests: Test ch6b_filetest in Process 65 exited with code 0
Ustests: Running ch6b_exec
argc = 6
argv[0] = (*o*)
argv[1] = (>.<)
argv[2] = (0.0)
argv[3] = (QwQ)
argv[4] = orz
argv[5] = 没有了呀
Ustests: Test ch6b_exec in Process 67 exited with code 0
Ustests: Running ch7b_pipetest
[parent] read end = 0x0000000000000003, write end = 0x0000000000000004
[parent] close read end
[parent] write over
[child] close write end
[chile] read over
Read OK, child process exited!
pipetest passed!
Ustests: Test ch7b_pipetest in Process 68 exited with code 0
ch7b Ustests passed!
Shell: Process 2 exited with code 0
>>
```

挺可爱的